

Formation complète Goose

Maîtriser un agent de développement puissant pour livrer plus vite, plus proprement et plus sûrement

Format pédagogique : modules, exercices, ateliers, checklists, projet final

Public visé : développeurs, tech leads, CTO, équipes produit/ingénierie souhaitant standardiser l'usage de Goose.

Prérequis : bases de CLI, Git et lecture de code.

Résultat attendu : savoir cadrer Goose, industrialiser des workflows, sécuriser l'exécution et transformer les meilleurs usages en standards d'équipe.

Durée recommandée : 4 semaines ou 24 à 30 heures de formation guidée.

Méthode : démonstration, pratique encadrée, exercices individuels, ateliers d'équipe.

Livrables : templates de prompts, fichiers de contexte, recettes, règles de sécurité et projet final.

1. Vue d'ensemble du parcours

Ce programme est conçu pour faire monter une équipe en compétence de façon progressive. Il démarre par les fondations d'usage, passe par la productivité quotidienne, puis évolue vers l'industrialisation, la sécurité et la standardisation des workflows.

Bloc	Objectif	Modules	Résultat
A. Fondations	Comprendre Goose et bien le cadrer	1 à 3	Passer d'un simple chat à un agent utilisable en production personnelle
B. Productivité	Travailler vite sur code, fichiers et CLI	4 à 6	Utiliser Goose pour analyser, modifier, automatiser et documenter
C. Industrialisation	Transformer les bons usages en standards	7 à 9	Construire recipes, règles de contexte et comportements reproductibles
D. Sécurité & scale	Réduire le risque et déployer en équipe	10 à 12	Sécuriser, gouverner et étendre l'usage à l'échelle

2. Compétences visées en fin de formation

- Structurer un prompt d'agent avec contexte, objectif, règles et validation.
- Utiliser les sessions, projets et mécanismes de gestion de contexte de Goose.
- Encadrer les modifications de code et d'outils avec des garde-fous clairs.
- Automatiser des workflows via la CLI et des fichiers d'instructions.
- Créer du contexte durable avec .goosehints, persistent instructions, skills et slash commands.
- Transformer un workflow gagnant en recipe réutilisable.
- Mettre en place une politique de sécurité : permissions, .gooseignore, adversary mode, sandbox.
- Déployer un cadre de travail d'équipe : standards, modèles de sortie, revue et certification.

Module 1 — Comprendre Goose et son modèle de travail

Objectif pédagogique : Comprendre ce que Goose sait faire, quand l'utiliser et comment le distinguer d'un simple assistant de chat.

Durée indicative : 2 h

À maîtriser	Production attendue
• Différence entre assistant conversationnel et agent outillé	• Un tableau d'usage de Goose par type de besoin

• CLI vs Desktop et cas d'usage de chacun • Rôle des extensions, outils, sessions et projets • Principaux scénarios : audit, implémentation, refactor, documentation, automatisation	• Une grille de décision “autonomie / supervision / refus”
--	--

Exercices et ateliers

Exercice 1.1 — Cartographier un dépôt existant

Sur un petit dépôt de test, demander à Goose de décrire l'architecture, les risques et les points de dette technique.

- Limiter l'analyse à un sous-répertoire précis.
- Exiger une restitution en 3 sections : architecture, risques, quick wins.
- Comparer la réponse de Goose avec votre propre lecture du code.

Livrable attendu : Un rapport d'analyse d'une page maximum.

Exercice 1.2 — Décider quand Goose est le bon outil

Étudier 6 tâches courantes et classer celles qui doivent passer par Goose, un IDE, un script ou un humain.

- Justifier le choix pour chaque tâche.
- Identifier au moins 2 tâches où Goose ne doit pas être autonome.

Livrable attendu : Tableau de décision “outil adapté par type de tâche”.

Checklist de validation

- ☐ Le périmètre est explicite.
- ☐ La sortie demandée est structurée.
- ☐ Les zones sensibles sont identifiées.

Erreurs fréquentes à éviter

- Lancer Goose sur une mission trop large sans périmètre.
- Lui demander de “tout corriger” sans règles de validation.
- Confondre rapidité et absence de contrôle.

Module 2 — Prompter Goose comme un développeur senior

Objectif pédagogique : Savoir rédiger des demandes claires, testables et orientées résultat.

Durée indicative : 2 h

À maîtriser • Structure recommandée : contexte, objectif, règles, validation • Formuler des contraintes techniques vérifiables • Demander un plan avant action risquée • Imposer un format de sortie exploitable	Production attendue • Un canevas de prompt standardisé • Une bibliothèque de 5 prompts de base
--	--

Exercices et ateliers

Exercice 2.1 — Réécrire trois prompts faibles

Prendre trois demandes vagues et les transformer en prompts exploitables.

- Ajouter le contexte projet.
- Ajouter au moins 3 contraintes de qualité.
- Ajouter une méthode de validation.

Livrable attendu : Trois prompts réécrits, chacun prêt à être utilisé.

Exercice 2.2 — Prompts de refactor sécurisé

Rédiger un prompt qui force Goose à proposer un plan avant modification.

- Interdire les changements d'API publique.
- Limiter les fichiers modifiables.
- Exiger les tests ciblés.

Livrable attendu : Un prompt de refactor réutilisable pour votre équipe.

Checklist de validation

- ☐ Le prompt contient un objectif mesurable.
- ☐ Les contraintes sont explicites.
- ☐ La validation est réalisable.

Erreurs fréquentes à éviter

- Prompts flous : “fais au mieux”, “refactor tout”.
- Absence de critères d'acceptation.
- Demander de modifier le code sans demander la liste des fichiers touchés.

Module 3 — Sessions, projets et continuité de travail

Objectif pédagogique : Comprendre comment Goose conserve, résume et exploite le contexte dans le temps.

Durée indicative : 2 h

À maîtriser • Créer, reprendre et cloisonner des sessions • Différence entre travail durable et session jetable • Gestion des limites de contexte et auto-compaction • Quand relancer une session au lieu de la prolonger	Production attendue • Une politique de nommage et d'archivage des sessions • Un guide de reprise de contexte
--	---

Exercices et ateliers

Exercice 3.1 — Créer trois sessions métier

Créer trois sessions nommées : bugfix-auth, refactor-billing et doc-onboarding.

- Définir pour chaque session son objectif, ses fichiers cibles et ses interdits.
- Comparer la qualité obtenue après reprise de session.

Livrable attendu : Une convention de nommage et de cycle de vie des sessions.

Exercice 3.2 — Nettoyer un contexte devenu bruité

Simuler une conversation trop longue et préparer une relance propre.

- Résumer l'état courant.
- Réémettre un prompt de reprise.
- Identifier ce qu'il ne faut pas reporter.

Livrable attendu : Template de reprise de session.

Checklist de validation

- ☐ La session correspond à un objectif cohérent.
- ☐ Le résumé de reprise tient en moins de 10 lignes.
- ☐ Les décisions importantes sont explicites.

Erreurs fréquentes à éviter

- Conserver une session malgré un changement complet de mission.
- Accumuler des consignes contradictoires.
- Reprendre une session sans résumer l'état utile.

Module 4 — Travailler proprement sur les fichiers et la codebase

Objectif pédagogique : Faire analyser et modifier du code par Goose sans mettre le dépôt en danger.

Durée indicative : 2 h 30

À maîtriser - Analyse d'impact avant modification - Stratégie de petits lots de changements - Limitation aux fichiers nécessaires - Relecture systématique du diff Git	Production attendue - Un protocole de modification sûre - Une checklist de revue de diff
---	---

Exercices et ateliers

Exercice 4.1 — Changement limité à deux fichiers

Demander une correction de bug avec interdiction de dépasser deux fichiers modifiés.

- Exiger la liste des fichiers avant écriture.
- Faire relire le diff final.
- Comparer l'implémentation obtenue à une solution manuelle.

Livrable attendu : Correctif Git revu et commenté.

Exercice 4.2 — Analyse d'impact avant refactor

Faire produire par Goose une carte des impacts d'un changement de signature de fonction.

- Identifier les appels directs et indirects.
- Lister les tests à exécuter.
- Proposer un plan par étapes.

Livrable attendu : Plan de refactor par étapes.

Checklist de validation

- ☐ Les fichiers cibles sont validés avant écriture.
- ☐ Le changement est incrémental.
- ☐ Le diff est lu avant validation.

Erreurs fréquentes à éviter

- Autoriser Goose à parcourir toute la base sans but.
- Fusionner un gros diff non relu.
- Ne pas vérifier les commandes shell exécutées.

Module 5 — Automatiser avec la CLI et goose run

Objectif pédagogique : Passer de l'usage interactif à des workflows automatisés et reproductibles.

Durée indicative : 2 h 30

À maîtriser • goose run avec texte, fichier d'instructions et stdin • Sessions nommées, reprise et mode sans session • Debug, verbosité et compréhension des appels d'outils • Cas d'usage CI, audit, documentation, migration	Production attendue • Un mini-catalogue de commandes CLI • Un workflow goose run réutilisable
---	--

Exercices et ateliers

Exercice 5.1 — Commande one-shot d'analyse

Écrire et exécuter une commande goose run pour auditer un projet.

- Utiliser -t pour une tâche courte.
- Structurer la sortie en markdown.
- Rendre la commande réutilisable.

Livrable attendu : Une commande commentée prête à l'emploi.

Exercice 5.2 — Instruction file pour audit sécurité

Créer un fichier instructions.md qui lance un audit de dépendances et produit un rapport.

- Ajouter étapes, contraintes et format de sortie.
- Tester le même workflow avec --debug.
- Comparer le résultat avec et sans session persistée.

Livrable attendu : Un fichier instructions.md versionnable.

Checklist de validation

- ☐ La commande est rejouable.
- ☐ Les paramètres importants sont visibles.
- ☐ Le résultat est exportable vers un artefact.

Erreurs fréquentes à éviter

- Automatiser un workflow non stabilisé.
- Oublier --no-session pour des scripts éphémères.
- Ne pas inspecter le mode debug lors de la mise au point.

Module 6 — Context engineering : .goosehints, skills, slash commands, persistent instructions

Objectif pédagogique : Construire le contexte durable qui fait gagner du temps à chaque session.

Durée indicative : 3 h

À maîtriser	Production attendue
• Utiliser .goosehints pour les règles projet • Utiliser persistent instructions pour les garde-fous permanents • Créer des skills orientés workflow • Créer des slash commands pour accélérer l'usage quotidien	• Un kit de contexte durable par projet • Une bibliothèque initiale de skills/commands

Exercices et ateliers

Exercice 6.1 — Concevoir un .goosehints de projet

Rédiger un fichier de contexte pour un projet réel ou fictif.

- Inclure architecture, conventions, règles de tests et zones interdites.
- Garder le document compact et actionnable.

Livrable attendu : Un .goosehints prêt à committer.

Exercice 6.2 — Créer 3 skills et 3 slash commands

Transformer des tâches répétitives en objets réutilisables.

- Définir le déclencheur, les étapes et le format de sortie.
- Créer au moins une commande pour revue de code et une pour génération de tests.

Livrable attendu : Un pack de skills et commandes réutilisables.

Checklist de validation

- ☐ Chaque élément a un rôle distinct.
- ☐ Les consignes sont stables dans le temps.
- ☐ Les commandes sont nommées de manière évidente.

Erreurs fréquentes à éviter

- Mettre trop de texte dans les instructions persistantes.
- Dupliquer le même contenu entre .goosehints et skills.
- Créer des commandes sans format de sortie.

Module 7 — Recipes : industrialiser les workflows gagnants

Objectif pédagogique : Transformer les meilleurs usages en workflows partageables et paramétrables.

Durée indicative : 2 h 30

À maîtriser	Production attendue
• Structure d'une recipe : outils, buts, paramètres, modèle, sortie • Quand créer une recipe plutôt qu'un prompt • Paramétrage avec variables • Organisation avec subrecipes et parallélisation raisonnée	• Une recipe standard d'équipe • Une convention de versionning des recipes

Exercices et ateliers

Exercice 7.1 — Créer une recipe de revue de code

Concevoir une recipe standard de code review pour votre stack.

- Définir des paramètres : langage, chemin, niveau de sévérité.
- Fixer un format JSON ou markdown de sortie.
- Décrire le public cible de la recipe.

Livrable attendu : Une recipe version 1.

Exercice 7.2 — Découper une recipe en subrecipes

Prendre un workflow trop gros et le scinder en sous-tâches : sécurité, tests, documentation.

- Définir l'entrée/sortie de chaque sous-étape.
- Décider ce qui peut tourner en parallèle ou non.

Livrable attendu : Un schéma d'orchestration de recipe.

Checklist de validation

- ☐ La recipe répond à un usage répété.
- ☐ Le format de sortie est stable.
- ☐ Les paramètres ont un vrai effet.

Erreurs fréquentes à éviter

- Créer une recipe trop générique.
- Multiplier les paramètres inutiles.

- Oublier de décrire les prérequis techniques.

Module 8 — Maîtriser les tools, permissions et sorties

Objectif pédagogique : Contrôler l'autonomie de Goose et réduire la surprise opérationnelle.

Durée indicative : 2 h

À maîtriser • Modes de permission et principe du moindre privilège • Ajustement de la sortie des outils • Quand utiliser un mode très supervisé ou plus autonome • Construction de profils d'usage par équipe	Production attendue • Une matrice permissions / usages • Une politique d'autonomie graduée
---	--

Exercices et ateliers

Exercice 8.1 — Définir trois profils de permission

Créer les profils Lecture seule, Développement encadré et Automatisation avancée.

- Lister les outils autorisés et interdits.
- Préciser le niveau de logs.
- Associer chaque profil à des cas d'usage.

Livrable attendu : Une matrice d'autorisations.

Exercice 8.2 — Concevoir une politique d'escalade

Décider quand Goose peut agir seul, quand il doit demander validation et quand il doit être bloqué.

- Inclure shell, écriture, suppression et réseau.
- Prévoir une règle spécifique pour les secrets.

Livrable attendu : Une politique d'escalade décisionnelle.

Checklist de validation

- ☐ Les permissions sont minimales.
- ☐ Les actions risquées nécessitent validation.
- ☐ Le niveau de logs est adapté.

Erreurs fréquentes à éviter

- Tout ouvrir “pour aller plus vite”.
- Ne pas journaliser les actions sensibles.

- Ne pas distinguer usage local, CI et production.

Module 9 — Personnalisation avancée : templates, variables d'environnement, configuration

Objectif pédagogique : Adapter Goose au contexte d'équipe et au type de tâches.

Durée indicative : 2 h

À maîtriser • Prompt templates pour la réponse, la planification et la compaction • Variables d'environnement liées au contexte, aux outils et aux modèles • Standardisation du comportement entre machines et environnements • Versionner les conventions utiles sans complexifier inutilement	Production attendue • Un standard de configuration d'équipe • Un guide de personnalisation minimaliste
--	---

Exercices et ateliers

Exercice 9.1 — Standard d'équipe de restitution

Définir une structure de réponse commune pour Goose.

- Titre, résumé, plan d'action, changements, validation, risques.
- Limiter le format à une page d'instructions.

Livrable attendu : Template de réponse d'équipe.

Exercice 9.2 — Fichier de configuration reproductible

Documenter les variables d'environnement et leurs valeurs de référence.

- Séparer valeurs locales, CI et postes sensibles.
- Justifier chaque variable retenue.

Livrable attendu : Un fichier de référence de configuration.

Checklist de validation

- ☐ Chaque réglage a une utilité claire.
- ☐ Les variables critiques sont documentées.
- ☐ La personnalisation reste compréhensible par un nouvel arrivant.

Erreurs fréquentes à éviter

- Personnaliser trop tôt sans retour d'expérience.
- Cacher des comportements critiques dans des réglages obscurs.
- Laisser diverger les configurations individuelles.

Module 10 — Sécurité opérationnelle : .gooseignore, adversary mode, sandbox

Objectif pédagogique : Travailler vite tout en limitant les risques de fuite, de suppression ou d'exécution non maîtrisée.

Durée indicative : 3 h

À maîtriser	Production attendue
• Bloquer l'accès aux secrets et zones sensibles avec .gooseignore • Utiliser adversary mode comme relecteur silencieux des appels d'outils • Utiliser le sandbox Desktop sur macOS lorsque pertinent • Formaliser une politique de sécurité agentique	• Une politique de sécurité agentique • Des fichiers de protection activables

Exercices et ateliers

Exercice 10.1 — Écrire un .gooseignore de production

Protéger les secrets, clés, répertoires critiques et artefacts sensibles.

- Inclure .env, clés privées, fichiers de configuration critiques.
- Préciser les motifs glob à éviter.

Livrable attendu : Un .gooseignore commenté.

Exercice 10.2 — Rédiger des règles adversary mode

Bloquer l'exfiltration, les suppressions hors périmètre et les scripts distants non fiables.

- Formuler des règles courtes et actionnables.
- Prévoir un comportement en cas de doute.

Livrable attendu : Un fichier adversary.md initial.

Checklist de validation

- ☐ Les secrets sont explicitement exclus.

- ☐ Les actions destructives sont contrôlées.
- ☐ Les règles sont testables.

Erreurs fréquentes à éviter

- Se contenter d'un discours de sécurité sans règles exécutables.
- Protéger les fichiers mais pas les commandes shell.
- Écrire des règles trop longues ou ambiguës.

Module 11 — Workflows avancés : subagents, providers et orchestration

Objectif pédagogique : Passer d'un usage individuel à une architecture de travail plus scalable.

Durée indicative : 2 h

À maîtriser	Production attendue
• Quand utiliser un subagent pour isoler une tâche • Séparer exploration, implémentation et validation • Choisir un provider ou un modèle selon la tâche • Orchestrer sessions, recipes et sous-tâches	• Une architecture cible d'usage avancé • Une matrice de choix de provider

Exercices et ateliers

Exercice 11.1 — Architecture d'un workflow à trois agents

Définir un workflow avec un agent d'audit, un agent d'implémentation et un agent de validation.

- Décrire l'entrée/sortie de chaque rôle.
- Préciser ce qui reste sous contrôle humain.

Livrable attendu : Un diagramme ou tableau d'orchestration.

Exercice 11.2 — Choix de provider par type de tâche

Comparer les besoins pour analyse longue, génération de code, revue sécurité et documentation.

- Relier chaque besoin à un profil de modèle.
- Lister les critères de choix.

Livrable attendu : Une matrice tâches ↔ modèle/provider.

Checklist de validation

- ☐ Chaque agent a un rôle unique.
- ☐ La supervision humaine est clairement placée.
- ☐ Le protocole de handoff est simple.

Erreurs fréquentes à éviter

- Lancer trop de sous-agents sans protocole.
- Mélanger exploration et implémentation dans la même boucle.
- Négliger le coût de coordination.

Module 12 — Projet final et certification

Objectif pédagogique : Assembler l'ensemble des acquis dans un cadre de travail réel ou quasi réel.

Durée indicative : 3 h

À maîtriser	Production attendue
• Cadrer un projet, le sécuriser, l'automatiser et le standardiser • Produire des artefacts réutilisables pour une équipe • Évaluer la maturité de l'usage de Goose • Construire un plan d'adoption à 30 jours	• Un kit d'équipe complet • Une feuille de route d'adoption

Exercices et ateliers

Exercice 12.1 — Pack de démarrage Goose pour une équipe

Sur un projet pilote, créer le kit complet de démarrage.

- Inclure .goosehints, persistent instructions, .gooseignore, 3 skills, 3 slash commands et 2 recipes.
- Définir la politique de validation humaine.
- Préparer la documentation de prise en main.

Livrable attendu : Un pack prêt à être partagé.

Exercice 12.2 — Soutenance finale

Présenter votre cadre d'usage, vos choix de sécurité et vos gains de productivité.

- Démontrer un workflow bout en bout.
- Montrer les garde-fous.
- Présenter la feuille de route d'adoption.

Livrable attendu : Une soutenance de 10 minutes et un dossier final.

Checklist de validation

- ☐ Les artefacts sont cohérents entre eux.
- ☐ Le workflow est démontré de bout en bout.
- ☐ Les risques et limites sont explicités.

Erreurs fréquentes à éviter

- Livrer des artefacts sans protocole d'usage.
- Ne pas mesurer les gains obtenus.
- Absence de règles de gouvernance après la formation.

Évaluation, certification et grille de notation

La certification interne ou personnelle peut être conduite à partir de la grille ci-dessous. Chaque critère est noté de 1 à 4.

Critère	1 — Débutant	2 — Fonctionnel	3 — Solide	4 — Maîtrise
Cadrage des prompts	Flou	Objectif partiel	Bien structuré	Reproductible et mesurable
Sécurité	Aucune règle	Règles implicites	Règles écrites	Règles exécutables et testées
Automatisation	Usage manuel uniquement	Quelques scripts	Workflows CLI stables	Recipes + standard d'équipe
Qualité des sorties	Inégale	Acceptable	Cohérente	Documentée, standardisée, audité
Conduite du changement	Aucune méthode	Expérimentation locale	Pilote d'équipe	Plan d'adoption et gouvernance

Annexe A — Canevas de prompt standard

- Contexte : repo, stack, fichiers concernés, historique utile.
- Objectif : résultat exact attendu.
- Règles : limites d'action, style, contraintes techniques, sécurité.
- Validation : tests à exécuter, format de sortie, niveau de preuve attendu.

Annexe B — Checklists opérationnelles

Avant d'autoriser une modification de code

- ☐ Le périmètre de fichiers est clair.
- ☐ Le plan a été validé.
- ☐ Les secrets et zones sensibles sont exclus.
- ☐ Les tests cibles sont définis.

Avant de passer en automatisation

- ☐ Le workflow a déjà réussi plusieurs fois à la main.
- ☐ La sortie attendue est stable.
- ☐ Les logs sont suffisants pour diagnostiquer un échec.
- ☐ Le mode sans session est utilisé si nécessaire.

Avant de diffuser à l'équipe

- ☐ Les conventions sont documentées.
- ☐ Les recipes sont versionnées.
- ☐ Les règles de sécurité sont activables.
- ☐ Un projet pilote a été validé.

Annexe C — Sources officielles à étudier

Cette formation a été structurée à partir des guides officiels Goose. Utilisez la liste ci-dessous comme parcours de lecture complémentaire.

- Guides — vue d'ensemble
- Managing Sessions
- Session Management
- In-Session Actions
- Smart Context Management
- Context Engineering
- Managing Tools
- Recipes
- Running Tasks
- CLI Commands
- Environment Variables